



(/rol/app/)

Menu

Korean

홈 (/rol/app/) (home) > Securing the OpenShift Service Mesh

> Chapter 1: Securing the OpenShift Service Mesh

Securing the OpenShift Service Mesh

Lesson

Lab Environment

Mutual TLS (mTLS) with OpenShift Service Mesh

Objectives

- Explain how mutual TLS provides transparent encryption and authentication by using SPIFFE identities and Envoy sidecar proxies.
- Configure peer authentication policies at mesh, namespace, workload, and port levels by using permissive, strict, or disable modes.
- Verify that mTLS configuration and troubleshoot common issues by using Kiali, certificate inspection, and istioctl validation.

Mutual TLS (mTLS) with OpenShift Service Mesh

Red Hat OpenShift Service Mesh provides a comprehensive security solution to protect microservice deployments through mutual TLS (mTLS). Unlike traditional TLS, where only servers prove their identity, mTLS requires both parties to authenticate with X.509 certificates, implementing a true zero-trust network model.

Traditional approaches to securing internal (east-west) communication in Kubernetes require organizations to manually provision and maintain their own Public Key Infrastructure (PKI). This manual approach requires cloud administrators to manage certificate authorities and distribute certificates. Additionally, application developers must modify code to use TLS protocols.

OpenShift Service Mesh eliminates this operational complexity by providing a managed PKI infrastructure. The service mesh achieves *security by default* without requiring application code changes. It automatically handles certificate provisioning, rotation, and TLS

termination at the sidecar proxy level. Applications communicate with their local Envoy proxy via plain HTTP, whereas the proxies handle encrypted mTLS communication between services.

This transparent encryption approach implements defense-in-depth security with the following critical benefits:

Strong identity foundation

Services receive first-class identities based on Kubernetes service accounts, encoded as Secure Production Identity Framework For Everyone (SPIFFE) IDs in X.509 certificates. These cryptographic identities remain stable even when pods are rescheduled or IP addresses change.

Zero-trust networking

All service-to-service communication is authenticated and encrypted by default, eliminating the need for implicit trust based on network location or IP addresses.

Defense against network attacks

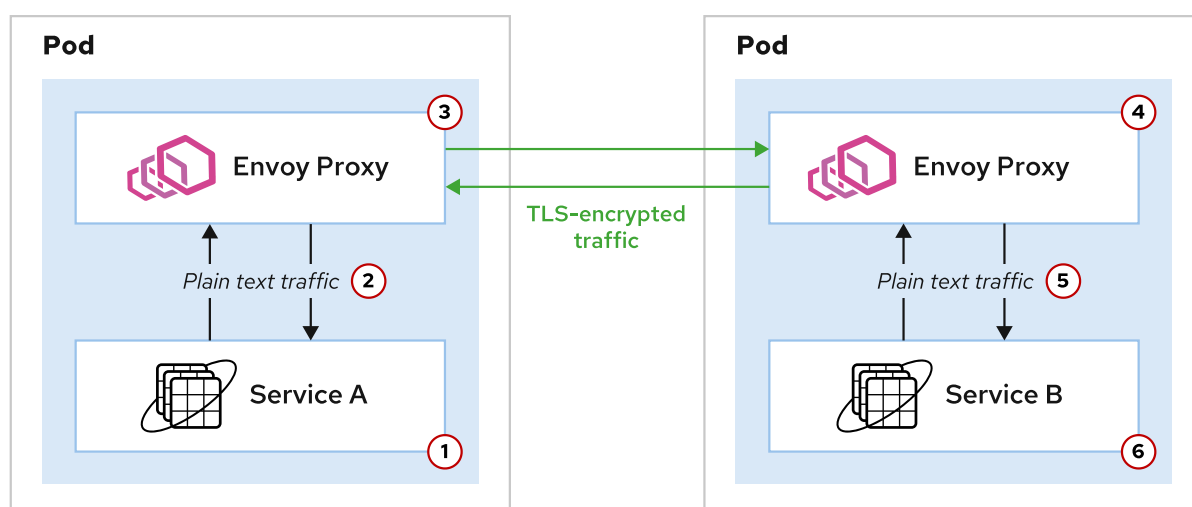
mTLS prevents man-in-the-middle attacks, eavesdropping, and service impersonation, even if an attacker gains access to the pod network. Traffic remains encrypted across the network, protecting against network-level threats. Refer to the references section for more information about cyberattacks.

Automated certificate management

Istiod provisions workload certificates, automatically rotates them before expiration, and manages the complete certificate lifecycle without manual intervention or service downtime.

How mTLS Works in the Service Mesh

OpenShift Service Mesh uses sidecar proxies as Policy Enforcement Points (PEPs) to transparently implement mTLS. When a service requests another service in the mesh, the communication follows this flow:



mTLS communication flow through Envoy proxies

- 1 The application sends a plain HTTP request to its local Envoy sidecar proxy.

- 2 The client-side Envoy proxy intercepts the request and initiates an mTLS connection to the destination service's Envoy proxy.
- 3 During the TLS handshake, both proxies exchange X.509 certificates to prove their identities.
- 4 The destination Envoy proxy validates the client's certificate and decrypts the traffic.
- 5 The decrypted request is forwarded to the destination application as plain HTTP
- 6 The response follows the same encrypted path in reverse.

This process happens transparently without changes to the application code. Applications are unaware that their traffic is encrypted because the encryption and decryption occur at the sidecar proxy layer.

Understanding SPIFFE Identities

Every workload in the service mesh receives a unique identity based on the SPIFFE standard. SPIFFE provides a framework for securely identifying software systems in dynamic and heterogeneous environments.

An SPIFFE ID follows this format:

```
spiffe://cluster.local/ns/NAMESPACE/sa/SERVICE_ACCOUNT
```

For example, the reviews service running in the bookinfo namespace with the bookinfo-reviews service account receives this identity:

```
spiffe://cluster.local/ns/bookinfo/sa/bookinfo-reviews
```

These identities are embedded in X.509 certificates and used during mTLS authentication. Unlike IP-based or network-based identification, SPIFFE IDs remain stable even when pods are rescheduled or IP addresses change.

Mutual TLS Modes

OpenShift Service Mesh supports three mTLS modes that control how services accept incoming traffic.

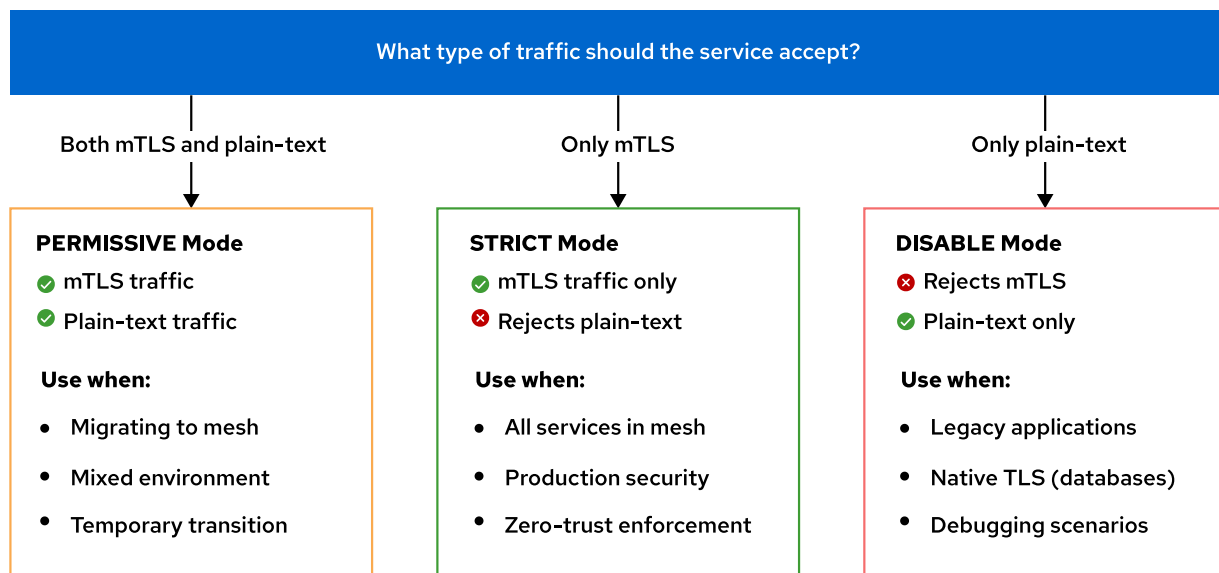


Figure 1.5: mTLS mode behavior comparison

Permissive Mode

Permissive mode is the default behavior when no peer authentication policy is configured. Services accept both mTLS-encrypted and plain-text traffic, enabling gradual migration without breaking existing connections.

Services with Envoy sidecars automatically establish mTLS connections when both endpoints support it, whereas services outside the mesh can send plain-text traffic. In production environments, permissive mode should be viewed as a temporary migration state rather than a permanent configuration.

Strict Mode

Strict mode enforces mTLS-only communication by rejecting all plain-text traffic. Services only accept connections from clients that present valid mTLS certificates with appropriate SPIFFE identities.

This mode provides the strongest security posture by ensuring all service-to-service communication is both encrypted and mutually authenticated. Any attempt to send plain-text traffic results in connection failures, typically appearing as *connection reset by peer* errors.

Strict mode is appropriate for production environments where all services are part of the mesh and have Envoy sidecars.

IMPORTANT

Mutual TLS alone is not sufficient for comprehensive security. Always combine strict-mode mTLS with authorization policies to control which authenticated services can access specific resources. mTLS provides authentication and encryption, but authorization policies enforce access control.

Disable Mode

Disable mode turns off mTLS enforcement entirely and accepts only plain-text communication. This mode is rarely used but can be necessary in specific scenarios, such as services that implement their own encryption (databases with native TLS), legacy applications that cannot function with mTLS-wrapped connections, or debugging scenarios where you need to inspect plain-text traffic.

Use disable mode sparingly and only for specific services or ports that have technical requirements preventing mTLS usage.

Configuring Mutual TLS with Peer Authentication

The peer authentication policy controls how services in the mesh accept incoming traffic. This resource determines whether services accept mTLS, plain-text, or both.

You can apply peer authentication policies at different scopes to control the reach of your mTLS enforcement.

Mesh-wide mTLS Configuration

To enforce mTLS across the entire service mesh, create a peer authentication policy in the root namespace (typically `istio-system`):

```
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
  namespace: istio-system 1
spec:
  mtls:
    mode: STRICT 2
```

¹ The root namespace where the mesh control plane is deployed

² The mTLS mode is applied to all workloads in the mesh

This mesh-wide policy applies to all namespaces and workloads in the service mesh.

Use mesh-wide strict mode only after verifying that all services have Envoy sidecars and can establish mTLS connections. Applying strict mode mesh-wide before all services are ready causes widespread connectivity failures.

Namespace-wide mTLS Configuration

To enforce mTLS for all services in a namespace, create a peer authentication policy without a selector:

```
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
  namespace: bookinfo 1
spec:
  mtls:
    mode: STRICT 2
```

- 1** The namespace where this policy applies
- 2** The mTLS mode must be one of PERMISSIVE, STRICT, or DISABLE

This policy applies to all workloads in the `bookinfo` namespace because no selector is specified. All services in the namespace reject plain-text traffic and only accept mTLS connections.

Workload-specific mTLS Configuration

To apply mTLS policies to specific workloads, use label selectors:

```
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: reviews-strict
  namespace: bookinfo
spec:
  selector:
    matchLabels:
      app: reviews 1
  mtls:
    mode: STRICT
```

- 1** This policy only applies to pods with the `app: reviews` label

This approach enables fine-grained control, enabling different services in the same namespace to have different mTLS requirements.

Port-level mTLS Configuration

Some services expose multiple ports with different security requirements. The following example resource configures mTLS on a per-port basis:

```

apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: mixed-mode
  namespace: bookinfo
spec:
  mtls:
    mode: STRICT ❶
  portLevelMtls:
    9080:
      mode: PERMISSIVE ❷
    9443:
      mode: DISABLE ❸

```

- ❶ Default mode for all ports
- ❷ Port 9080 accepts both mTLS and plain-text traffic
- ❸ Port 9443 only accepts plain-text traffic (mTLS disabled)

NOTE

Port numbers in `portLevelMtls` refer to the workload's container port, not the Kubernetes service port. Specify the actual port the application listens on, not the service's exposed port.

Policy Precedence and Inheritance

When multiple peer authentication policies could apply to a workload, OpenShift Service Mesh applies them in this precedence order:

1. *Port-level settings*, the most specific
2. *Workload-level policies*
3. *Namespace-level policies*
4. *Mesh-wide policies*, the least specific

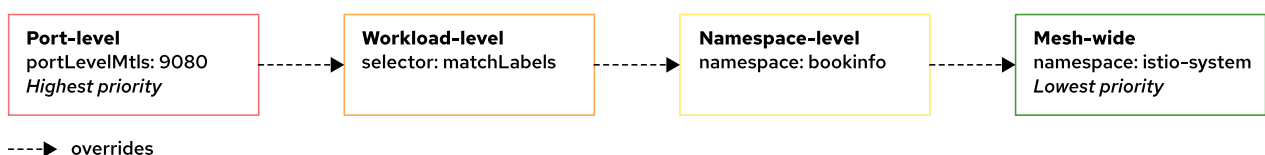


Figure 1.6: Peer authentication policy precedence hierarchy

More specific policies always override broader policies, enabling fine-grained control over mTLS enforcement. For example, a port-level permissive setting overrides a namespace-level strict policy for that specific port.

When a policy does not explicitly set a mode, it inherits from the next broader policy in the hierarchy. If no policies exist at any level, then the default mode is permissive and accepts both mTLS and plain-text traffic.

Configuring Client-side mTLS with Destination Rule

The *peer authentication* policy controls how services *accept* incoming traffic, and the *destination rule* resource controls how services *send* outgoing traffic. Both resources work together to establish end-to-end mTLS communication.

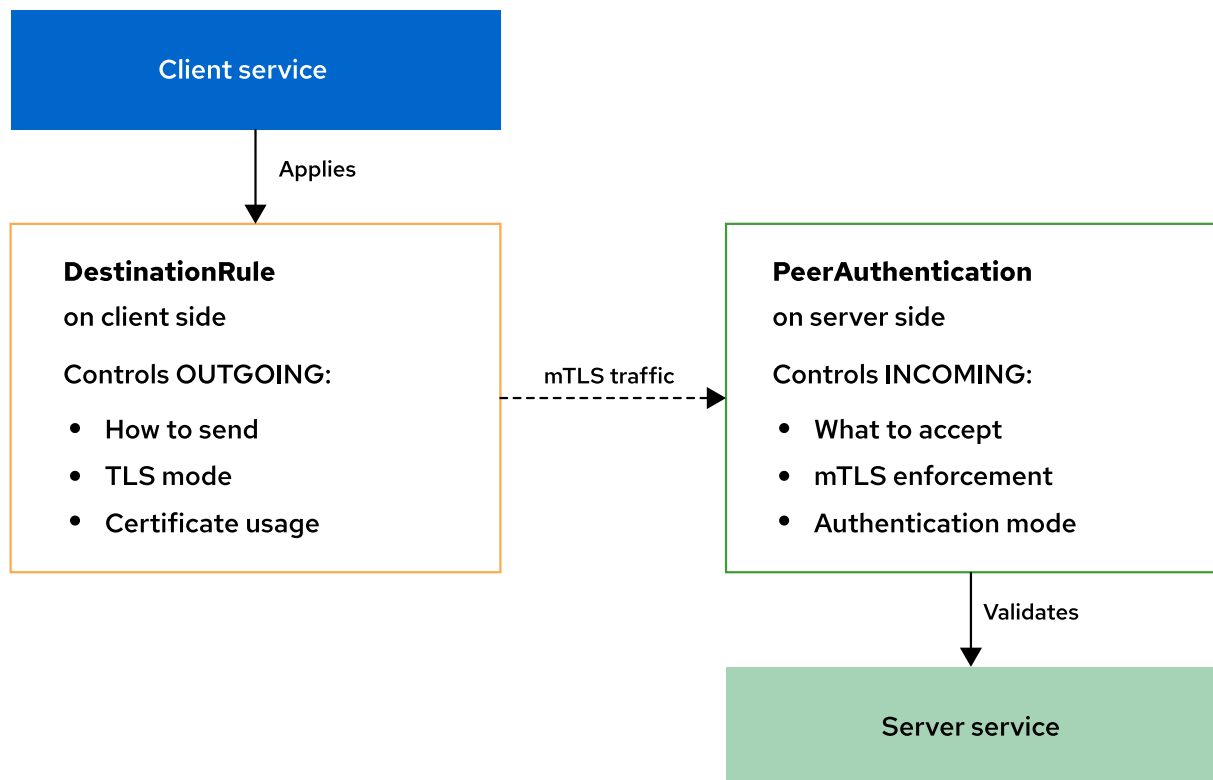


Figure 1.7: Peer authentication and destination rule Relationship

By default, OpenShift Service Mesh is configured with the `Auto mTLS` property enabled. This automatically uses mTLS between services with sidecars without requiring explicit destination rule configuration.

Create destination rule resources only when you need to customize TLS settings beyond the defaults or configure connections to external services. To explicitly configure client-side mTLS behavior, create a destination rule:


```

apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: default
  namespace: bookinfo
spec:
  host: "*.bookinfo.svc.cluster.local" ❶
  trafficPolicy:
    tls:
      mode: ISTIO_MUTUAL ❷

```

❶ This rule applies to all services in the bookinfo namespace

❷ Use Istio's automatic mTLS with Istio-managed certificates

The mode field in destination rule resource supports four TLS modes:

- DISABLE: No TLS connection to upstream endpoints
- SIMPLE: Standard TLS with server-only authentication (for external HTTPS services)
- MUTUAL: Manual mTLS requiring explicit certificate configuration (clientCertificate, privateKey, caCertificates)
- ISTIO_MUTUAL: Automatic mTLS by using Istio-managed certificates (recommended for mesh services)

Verifying and Validating mTLS Configuration

OpenShift Service Mesh provides different options to verify that mTLS is operating correctly.

Visual Verification with Kiali

When you enable the **Display** → **Security** option, the Kiali traffic graph provides visual confirmation of mTLS:

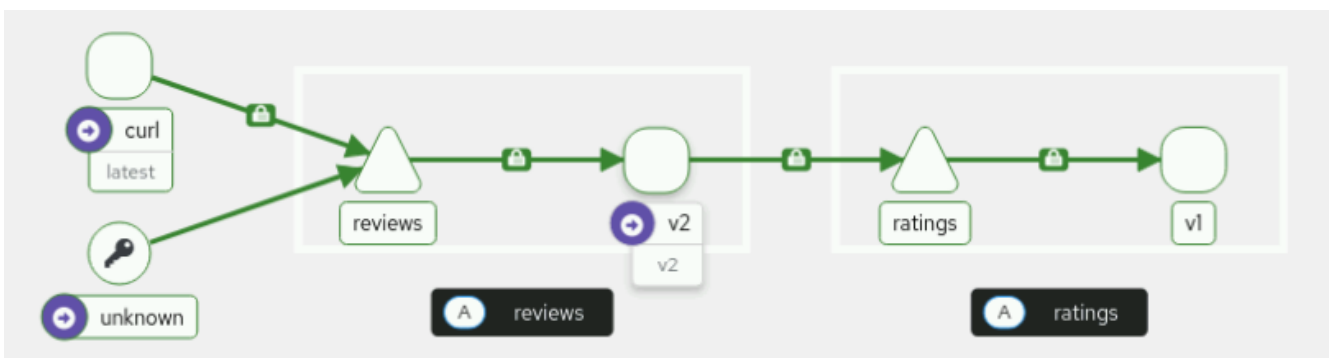


Figure 1.8: Kiali traffic graph with mTLS

- *Padlock icons* appear on edges between services by using mTLS
- *No padlock icon* indicates plain-text traffic
- Click on edges to view SPIFFE identities for source and destination

This visual feedback helps you quickly identify which connections are encrypted.

Inspecting Certificates

View certificate details for a workload:

```
[user@host ~]$ istioctl proxy-config secret POD_NAME
```

RESOURCE NAME	TYPE	STATUS	VALID CERT	SERIAL NUMBER	NOT AFTER	NOT BEFORE
default	Cert Chain	ACTIVE	true	abc123...	...	...
ROOTCA	CA	ACTIVE	true	def456...	...	...

The output shows the workload's certificate chain, root CA certificate, validity periods, and serial numbers.

Configuration Validation

Before deploying changes, use `istioctl analyze` to detect configuration problems:

```
[user@host ~]$ istioctl analyze
```

Warn [IST0102] (Namespace default) The namespace is not enabled for Istio

This command validates your configuration and identifies issues such as conflicting mTLS modes, invalid selectors, and missing resources.

Certificate Management

OpenShift Service Mesh automatically manages the complete lifecycle of workload certificates without manual intervention. Istiod acts as the Certificate Authority (CA) for the mesh, issuing certificates to workloads based on their Kubernetes service account identities.

By default, Istiod generates a self-signed root certificate and key during installation. This root CA signs all workload certificates in the mesh, establishing a chain of trust.

IMPORTANT

For production environments, Red Hat strongly recommends a production-ready CA such as HashiCorp Vault rather than self-signed certificates. Organizations can also use their own root and intermediate certificates to integrate with existing PKI infrastructure.

Istio uses a multi-component architecture to securely and automatically provision certificates. The certificate provisioning architecture involves three components working together:

- *Istiod*: Acts as the Certificate Authority, validating CSRs and signing certificates
- *Istio agent*: Runs alongside Envoy in each pod, generates CSRs, manages private keys, and serves certificates to Envoy
- *Envoy proxy*: Requests certificates from the Istio agent and uses them for mTLS connections

When a workload pod starts, the following certificate provisioning workflow occurs:

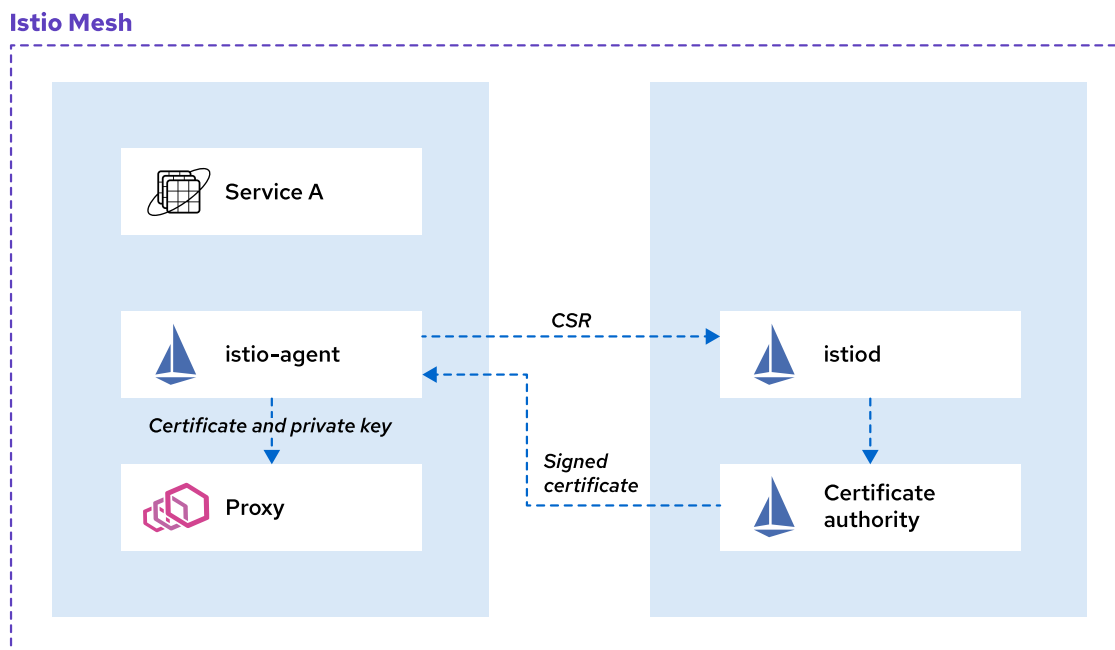


Figure 1.9: Automatic certificate provisioning workflow

1. Istiod exposes a gRPC service that accepts certificate signing requests (CSRs)
2. The Istio agent generates a private key and creates a CSR, then sends the CSR with the pod's credentials to Istiod for signing
3. The Certificate Authority in Istiod validates the credentials carried in the CSR, and upon successful validation, signs the CSR to generate the certificate
4. When the Envoy proxy starts, it requests the certificate and key from the Istio agent in the same container through the Envoy Secret Discovery Service (SDS) API
5. The Istio agent delivers the signed certificate (received from Istiod) and the private key to Envoy through the SDS API
6. The Istio agent monitors the certificate expiration, and the process repeats periodically for automatic certificate rotation

This architecture ensures that private keys never leave the workload pod. The Istio agent acts as a secure intermediary, managing the CSR process with Istiod and serving certificates to Envoy through the SDS API. The pod's Kubernetes service account determines its SPIFFE identity, ensuring that the cryptographic identity aligns with Kubernetes RBAC principles.

Certificates issued by Istiod have short lifespans (typically 24 hours) to limit the impact of potential compromise. The Istio agent monitors certificate expiration and automatically initiates rotation before certificates expire, ensuring continuous mTLS protection.

The rotation process is completely transparent to applications. Active connections continue with the existing certificate whereas new connections use the rotated certificate. This zero-downtime approach ensures that security maintenance occurs automatically without

requiring service restarts or manual intervention.

Securing Ingress Traffic with mTLS

External clients can also connect to mesh services by using TLS through the ingress gateway. The ingress gateway serves as the entry point for external traffic, terminating TLS before routing requests to internal services.

The Gateway OpenShift Service Mesh resource supports two primary TLS modes for external traffic:

Simple Mode (Server-side TLS)

The gateway presents a server certificate to clients, providing standard HTTPS encryption. Clients verify the gateway's certificate but do not present their own certificates. This mode is suitable for typical web applications where server authentication is sufficient.

Mutual Mode (Client Certificate Authentication)

Both the gateway and external clients exchange certificates for mutual authentication. Clients must present valid certificates signed by a trusted CA to access mesh services. This mode provides stronger security for machine-to-machine communication or for API access that requires client authentication.

The gateway stores certificates in Kubernetes secrets and uses the `credentialName` field to reference them. After TLS termination at the gateway, traffic flows to backend services by using the mesh's internal mTLS configuration.

The following example configures a Gateway resource with mutual TLS for external clients:

```
apiVersion: networking.istio.io/v1
kind: Gateway
metadata:
  name: bookinfo-gateway
spec:
  selector:
    istio: ingressgateway
  servers:
  - port:
      number: 443
      name: https
      protocol: HTTPS
    tls:
      mode: MUTUAL ❶
      credentialName: bookinfo-credential ❷
    hosts:
      - "bookinfo.example.com" ❸
```

- ❶ Mutual mode requires clients to present valid certificates
- ❷ References a Kubernetes secret containing the server certificate, key, and CA certificate for client validation
- ❸ The hostname that this gateway configuration applies to

NOTE

Ingress gateway TLS configuration differs from peer authentication policies, which control service-to-service communication within the mesh. Gateway TLS handles external client connections, whereas peer authentication policy manages internal workload-to-workload traffic.

Troubleshooting Mutual TLS Issues

Understanding common mTLS problems helps you quickly diagnose and resolve connectivity issues.

Connection reset by peer errors after enabling strict mode

Cause: Client without Envoy sidecar sending plain-text traffic.

Solution: Verify client pod has sidecar (2/2 ready) and namespace is in mesh, temporarily use permissive mode.

`istioctl analyze` reports policy conflicts

Cause: Multiple peer authentication policies with overlapping selectors.

Solution: Review all policies, avoid overlapping scopes, use workload-specific selectors.

TLS handshake failures in Envoy logs

Cause: Certificate trust chain issues or clock skew.

Solution: Verify Istiod is healthy, confirm NTP synchronization, restart pods for fresh certificates, and examine logs.

Health probe failures after strict mTLS

Cause: Kubernetes probes do not support mTLS.

Solution: Configure port-level permissive/disable mode on health check ports.

Migration Strategy for Production Environments

When adopting mTLS in production, follow a phased, incremental approach to minimize risk and service disruption.

IMPORTANT

Before beginning migration, verify that all services can communicate successfully without mTLS. Test baseline connectivity to establish a known-good state before introducing encryption policies.

Phase 1: Add Services to the Mesh Without Policy Changes

Enable sidecar injection on target namespaces, roll out deployments, and verify all pods show 2/2 ready status (application + Envoy sidecar). Services automatically use mTLS where both endpoints are in the mesh (opportunistic mTLS) and can accept plain-text

traffic from services outside the mesh.

Phase 2: Monitor Automatic mTLS Adoption

Use Kiali to visualize mTLS connections (padlock icons) and identify services without mTLS. Ensure that all client applications are in the mesh before proceeding to strict enforcement.

Phase 3: Enable Strict Mode Per Namespace

Start with non-critical namespaces, create peer authentication policy with mode: STRICT, monitor for connection failures, and test thoroughly. If issues arise, then switch back to permissive mode by updating the policy. Deploy incrementally to limit the potential impact of issues.

Phase 4: Expand to Mesh-Wide Enforcement

After validating pilot namespaces, expand strict mode incrementally. Eventually enable mesh-wide strict mode by creating a peer authentication policy in the `istio-system` namespace. Consider implementing authorization policies to add access control beyond authentication and encryption.

WARNING

If not all services can be migrated to the service mesh, you must keep those services in permissive mode. Document which services require permissive mode and why.

Ongoing Security Maintenance

Maintain security configuration through regular operational tasks, including the following:

- Audit peer authentication policies across namespaces
- Monitor certificate expiration and rotation
- Review which services remain in permissive mode and establish migration timelines
- Track mTLS adoption metrics to identify services without encryption
- Stay on supported OpenShift Service Mesh releases to receive security updates

IMPORTANT

Although mTLS provides essential security, it introduces computational overhead for encryption and decryption operations. Monitor service latency and CPU usage when implementing mTLS, especially for high-throughput services. The Envoy proxy efficiently handles TLS operations, but performance testing is recommended for latency-sensitive applications.

REFERENCES

Istio.io: Istio Security Concepts (<https://istio.io/v1.26/docs/concepts/security/>)

Istio.io: Mutual TLS Migration

(<https://istio.io/v1.26/docs/tasks/security/authentication/mtls-migration/>)

Istio.io: PeerAuthentication API Reference

(https://istio.io/v1.26/docs/reference/config/security/peer_authentication/)

Istio.io: Istio Security Best Practices (<https://istio.io/v1.26/docs/ops/best-practices/security/>)

Istio.io: Plugging in External CA Certificates

(<https://istio.io/v1.26/docs/tasks/security/cert-management/plugin-ca-cert/>)

Istio.io: Secure Gateways (<https://istio.io/v1.26/docs/tasks/traffic-management/ingress/secure-ingress/>)

SPIFFE Overview (<https://spiffe.io/docs/latest/spiffe-about/overview/>)

IBM Think: Cyberattacks (<https://www.ibm.com/think/topics/cyber-attack#498277090>)

IBM Think: Man-in-the-middle attack

(<https://www.ibm.com/think/topics/man-in-the-middle>)

Revision: ad00091-3.1-ed3-20260112-486e433



개인 정보 취급 방침 (https://www.redhat.com/ko/about/privacy-policy?extIdCarryOver=true&sc_cid=701f20000001D8QoAAK)

이용 약관 (<https://www.redhat.com/ko/about/terms-use>)

릴리스 노트

(https://docs.redhat.com/en/documentation/red_hat_learning_subscription/1-latest/html-single/red_hat_learning_subscription_release_notes/index)

Red Hat 교육 정책

(<https://www.redhat.com/ko/about/red-hat-training-policies>)

모든 정책 및 지침

(<https://www.redhat.com/ko/about/all-policies-guidelines>)

쿠키 기본설정